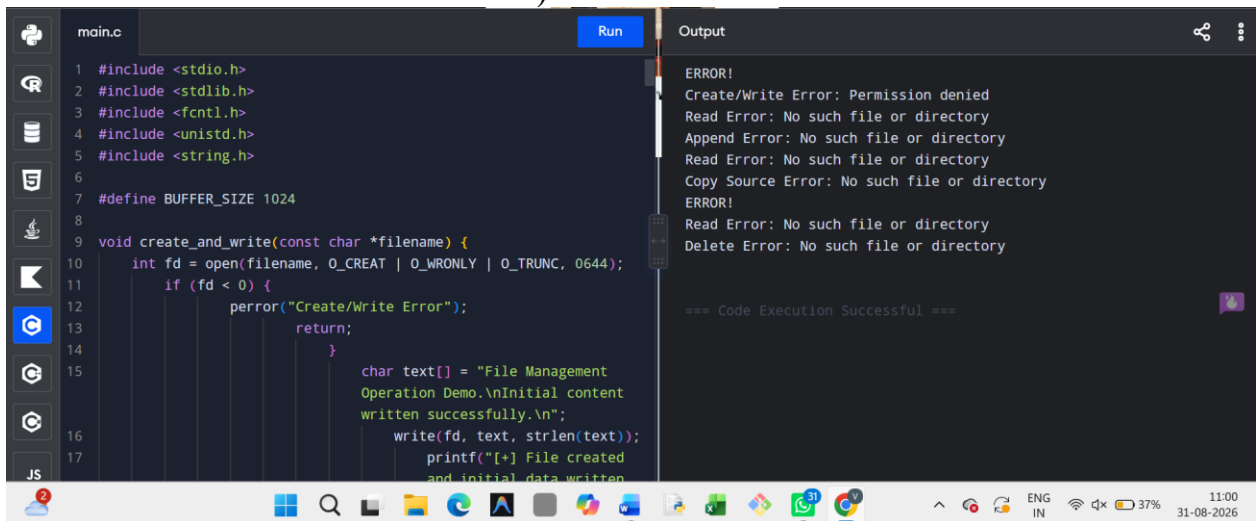


EXPERIMENT 36:

36. Construct a C program to implement the file management operations (Create, Write, Read, Append, Copy, and Delete).



The screenshot shows a C program in a code editor with the following code:

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 #include <string.h>
6
7 #define BUFFER_SIZE 1024
8
9 void create_and_write(const char *filename) {
10     int fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
11     if (fd < 0) {
12         perror("Create/Write Error");
13         return;
14     }
15     char text[] = "File Management\nOperation Demo.\nInitial content\nwritten successfully.\n";
16     write(fd, text, strlen(text));
17     printf("[+] File created\nand initial data written\n");
18 }
```

The output window shows the following messages:

```
Output
ERROR!
Create/Write Error: Permission denied
Read Error: No such file or directory
Append Error: No such file or directory
Read Error: No such file or directory
Copy Source Error: No such file or directory
ERROR!
Read Error: No such file or directory
Delete Error: No such file or directory

=== Code Execution Successful ===
```

PSEUDO CODE :

- Create & Write: Open file source.txt using open() with flags O_CREAT | O_WRONLY | O_TRUNC. Write initial text data to it using write().
- Read: Open source.txt with O_RDONLY. Read contents into a buffer using read() and display to standard output.
- Append: Open source.txt with O_WRONLY | O_APPEND. Write additional text data to the end of the file.
- Copy: Open source.txt for reading and destination.txt for writing (O_CREAT | O_WRONLY | O_TRUNC). Read chunks of data from source.txt and write them into destination.txt.
- Delete: Remove destination.txt from the filesystem using unlink() or remove()

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#define BUFFER_SIZE
```

```
1024
```

```
void
```

```
create_and_write(const
```

```
char *filename) {
```

```
    int fd =
```

```
    open(filename,
```

```
    O_CREAT |
```

```
    O_WRONLY |
```

```
    O_TRUNC, 0644);
```

```
    if (fd < 0) {
```

```
        perror("Create/Write
```

```
        Error");
```

```
        return;
```

```
    }
```

```
char text[] = "File  
Management Operation  
Demo.\nInitial content  
written successfully.\n";
```

```
write(fd, text,  
strlen(text));
```

```
printf("[+] File  
created and initial data  
written to '%s'.\n",  
filename);
```

```
close(fd);  
  
}
```

```
void read_file(const  
char *filename) {
```

```
int fd =  
open(filename,  
O_RDONLY);
```

```
if (fd < 0) {  
  
perror("Read  
Error");
```

```
        return;

    }

    char
buffer[BUFFER_SIZE];

    int bytesRead;

    printf("\n--- Reading
Content of '%s' ---\n",
filename);

    while ((bytesRead =
read(fd, buffer,
BUFFER_SIZE - 1)) >
0) {

        buffer[bytesRead]
= '\0';

        printf("%s",
buffer);

    }

    printf("-----
-----\n");

    close(fd);
```

```
}
```

```
void append_file(const  
char *filename) {
```

```
    int fd =
```

```
    open(filename,
```

```
    O_WRONLY |
```

```
    O_APPEND);
```

```
    if (fd < 0) {
```

```
        perror("Append  
Error");
```

```
        return;
```

```
    }
```

```
    char append_text[] =  
    "Appended line: Adding  
new data to existing  
file.\n";
```

```
    write(fd, append_text,  
    strlen(append_text));
```

```
    printf("[+] Appended
```

```
data to '%s'.\n",
```

```
filename);
```

```
close(fd);
```

```
}
```

```
void copy_file(const
```

```
char *src_file, const
```

```
char *dest_file) {
```

```
int src_fd =
```

```
open(src_file,
```

```
O_RDONLY);
```

```
if (src_fd < 0) {
```

```
    perror("Copy  
Source Error");
```

```
    return;
```

```
}
```

```
int dest_fd =
```

```
open(dest_file,
```

```
O_CREAT |
```

```
O_WRONLY |
```

```
O_TRUNC, 0644);
```

```
if (dest_fd < 0) {
```

```
    perror("Copy Dest  
Error");
```

```
    close(src_fd);
```

```
    return;
```

```
}
```

```
char
```

```
buffer[BUFFER_SIZE];
```

```
int bytesRead;
```

```
while ((bytesRead =  
read(src_fd, buffer,  
BUFFER_SIZE)) > 0) {
```

```
    write(dest_fd,  
buffer, bytesRead);
```

```
}
```

```
printf("[+] Copied  
contents from '%s' to
```

```
'%s'.\n", src_file,  
dest_file);
```

```
close(src_fd);
```

```
close(dest_fd);
```

```
}
```

```
void delete_file(const  
char *filename) {
```

```
    if (unlink(filename)  
== 0) {
```

```
        printf("[+] File '%s'  
successfully deleted.\n",  
filename);
```

```
    } else {
```

```
        perror("Delete  
Error");
```

```
    }
```

```
}
```



```
int main() {  
  
    const char *file1 =  
    "demo_source.txt";  
  
    const char *file2 =  
    "demo_copy.txt";  
  
    // Perform file  
operations sequentially  
  
    create_and_write(file1);  
  
    read_file(file1);  
  
    append_file(file1);  
  
    read_file(file1);  
  
    copy_file(file1, file2);  
  
    read_file(file2);  
  
    delete_file(file2);  
}
```

```
return 0;
```

}INPUT :

(No direct user input
required; file paths are
processed internally)

OUTPUT:

[+] File created and
initial data written to
'demo_source.txt'.

--- Reading Content of
'demo_source.txt' ---

File Management
Operation Demo.

Initial content written
successfully.

[+] Appended data to

'demo_source.txt'.

--- Reading Content of

'demo_source.txt' ---

File Management

Operation Demo.

Initial content written

successfully.

Appended line: Adding
new data to existing file.

[+] Copied contents

from 'demo_source.txt'

to 'demo_copy.txt'.

--- Reading Content of

'demo_copy.txt' ---

File Management

Operation Demo.

Initial content written

successfully.

Appended line: Adding
new data to existing file.

[+] File 'demo_copy.txt'

successfully deleted.